Python Introduction and Evolution

Study guide | Python 3 | Spark-free | Databricks-ready examples

1. Introduction

Python is a readable, general-purpose programming language created by Guido van Rossum. It was first publicly released in 1991; Python 1.0 followed in 1994, Python 2.0 in 2000, and Python 3.0 in 2008. Python 2 reached end of life in 2020. Python 3 is the modern supported line, and CPython is its most widely used implementation. Data engineers use Python for ingestion, validation, orchestration, APIs, and analytics.

2. Syntax

Python uses indentation to define blocks, names are case-sensitive, and statements are usually one per line. A name is bound with `=`. The interpreter executes code, while a notebook runs code in cells. Check the Python version and installed packages supplied by the Databricks Runtime before depending on a feature.

```
import sys
print(sys.version)
record_count = 3
if record_count:
    print(f"Ready to process {record_count} records")
```

3. Real-world data example

A scheduled ingestion task may read a small API response, validate required fields, transform dates and amounts, and log accepted/rejected counts. Python can coordinate these steps; the storage and compute choice depends on data size and platform.

4. Various ways of using it

- Run a `.py` script as a job for repeatable batch logic.
- Use a Databricks notebook for interactive, documented demonstrations and diagnostics.
- Organize production logic into importable modules and test functions independently.

5. Best practices

- Use a supported Python 3 runtime and document important dependency/version assumptions.
- Keep notebook cells reproducible and avoid relying on hidden execution order.
- Separate configuration, transformation logic, and side effects.

6. Do's

- Do check `sys.version` and package availability in the target runtime.
- Do make sample data deterministic and keep credentials outside source code.

7. Don'ts

- Don't assume a notebook variable exists because it was set in a previous interactive session.
- Don't treat Python version numbers or package availability as identical across runtime images.

8. Pros

- Readable syntax and a broad standard-library and third-party ecosystem.
- Works well for glue code, validation, APIs, and data tooling.

9. Cons

- Dynamic typing requires validation and testing.
- Pure Python is not the fastest option for every compute-intensive workload.

10. Practice examples with solutions

Try each task before reviewing its solution. Each example uses local, synthetic data and requires no external credentials.

Practice 1: Print the interpreter's major and minor version.

Solution

```
import sys
print(f"{sys.version_info.major}.{sys.version_info.minor}")
```

Practice 2: Count records and report whether the batch is empty.

Solution

```
rows = [{"id": 1}, {"id": 2}]
print(f"count={len(rows)}, empty={not rows}")
```

Practice 3: Write a function-shaped batch status message using an f-string.

Solution

```
batch_id, accepted, rejected = "B-42", 18, 2
print(f"{batch_id}: accepted={accepted}, rejected={rejected}")
```

11. Databricks notes

Import the companion `.py` file into Databricks as a source notebook. The demonstrations are Spark-free and use embedded synthetic data. pandas examples require pandas in the selected runtime; standard Databricks runtimes commonly include it. Confirm runtime versions and policies before using optional packages in production.